CSEN 296B-2 | Week 6.1 Lab Submission

Connecting Design to Prototype - AstraNotes Realization Slices

Student Name: Atishay Jain | **Date:** May 10, 2026 | **Project:** AstraNotes | **Technical Path:** Python

This is the first week I am writing code that a user could actually run. I took two slices that have been waiting in the backlog since Week 2.2 (US-01 create a note, US-05 list and load on startup) and wired them through the layered architecture that the Week 4.2 UML package committed to. The Sprint Zero skeleton already had the entity, repository adapter, privacy service, and validation layer; the missing pieces were the *NoteManager* orchestrator and a runnable CLI shell that ties the layers together. Everything below sits inside that frame - I did not introduce new components, new dependencies, or new requirements.

Development Direction

Decision	Choice	Reason
Language	Python 3.10+	Locked in Week 1.2 (ADR-001). Every diagram, test, and dependency I have already submitted assumes it.
App form	Local desktop CLI (*python -m astranotes*)	Already committed in five places: Sprint Zero plan, use case diagram, activity diagram, deployment diagram (*cli/* subgraph), and CLAUDE.md. A GUI or TUI this week would contradict the Week 4.2 UML package.
UI library	None - plain *input()* / stdout	Matches the deployment diagram. Avoids adding a third-party UI dep that has not been vetted under SPR-04.
Encryption library	cryptography 44.0.0 (Fernet)	Already in *pyproject.toml*, already documented under SPR-01 and SPR-04. No change.

The rubric mentions menu/profile/settings/notes as UI areas to make decisions about. I read that as a checklist of *decisions*, not a checklist of *screens* - the professor's wording is 'layout decisions'. Since AstraNotes is single-user with no accounts (a Sprint Zero design call), I do not ship a profile screen. The Settings screen states this explicitly so the omission is visible rather than silent.

Project Structure and Run Environment

The Sprint Zero layout is the same one the Week 4.2 deployment diagram drew. Week 6 added one new module (*services/note_manager.py*) and filled in the previously-empty *cli/* folder.

```
astranotes/
  __main__.py               # entry point: python -m astranotes
  cli/
    app.py                  # NEW this week - text menu shell
  models/
    note.py                 # Note dataclass            (Sprint Zero)
    exceptions.py           # AstraNotesError family    (Sprint Zero)
  repositories/
    base.py                 # NoteRepository ABC        (Sprint Zero)
    json_file.py            # JsonFileRepository        (Sprint Zero)
  services/
    note_manager.py         # NEW this week - orchestrator
    privacy.py              # PrivacyService (Fernet)   (Sprint Zero)
    validation.py           # ValidationLayer           (Sprint Zero)
tests/
  test_note.py, test_repository.py, test_privacy.py,
  test_validation.py        # 21 tests from Sprint Zero, still passing
```

```
  test_note_manager.py       # NEW - 8 tests
  test_cli.py                # NEW - 5 tests
pyproject.toml               # cryptography==44.0.0, pytest==8.3.4 [dev]
```

Run environment. macOS / Linux / Windows with Python 3.10+ and a virtualenv. Install with *pip install -e ".[dev]"*. Tests run with *pytest* (no external services; all I/O goes through *tmp_path* fixtures per SPR-03). The app runs with *python -m astranotes*. Data directory is *~/.astranotes/notes* by default, overridable via *ASTRANOTES_DATA_DIR*. The Fernet key is read from *ASTRANOTES_KEY*; if it is not set, the app prints a visible warning and generates a session-only key so private notes still work for a demo run - the persistent-key storage decision is the open SPR-01 ADR.

UI Shell

The CLI presents one main menu:

```
AstraNotes - main menu
  1) Create a note
  2) List notes
  3) Settings
  4) About
  5) Quit
Choose [1-5]:
```

Notes workspace = options 1 and 2. Option 1 prompts for title, body, and a y/N for *is_private*, then calls *NoteManager.create_note*. Option 2 calls *NoteManager.list_notes* and prints each note in *modified_at* descending order with a *[private]* flag and a 60-char snippet.

Settings = option 3. Prints the resolved data directory, the privacy key source (*ASTRANOTES_KEY* env var or 'dev key (session only)'), and an explicit note that the profile screen is intentionally omitted.

About = option 4. App name, version, course, and one-line summary of the storage and privacy rules. **Quit** = option 5.

All four behavioral paths are covered by *tests/test_cli.py*. *run()* takes *inp* and *out* as parameters so the tests drive it with a *StringIO* and an iterator of scripted answers - no real keyboard or screen involved. That dependency-injection seam is exactly what NFR-02 asked for.

First Functionality Slices

Slice 1 - Create a note (US-01, FR-01, FR-04, SPR-01)

NoteManager.create_note(title, body, is_private, tags) does four things in order:

1. Build a fresh Note dataclass - UUID and timestamps come from the entity's default factories (FR-01, FR-06).

2. *ValidationLayer.validate(note)* - raises *ValidationError* on whitespace-only titles before anything touches disk.

3. If *is_private* is true, encrypt the body with *PrivacyService.encrypt* and store a copy of the Note with the ciphertext body. If *is_private* is true and no *PrivacyService* was injected, raise *PersistenceError* rather than silently downgrading. The plaintext stays in the return value so the caller sees the readable note.

4. *JsonFileRepository.save(stored)* - one *{uuid}.json* file per note.

This slice realizes the exact left-hand and middle columns of *docs/uml/activity-diagram.mmd* (the 'Create a private note' workflow). The validation gate, the *is_private?* branch, and the encrypt-before-save ordering are all preserved.

Slice 2 - List and load on startup (US-05, FR-05, FR-07, FR-08, SPR-02)

NoteManager.list_notes() walks the repository, decrypts private notes that have a working key, and sorts the result.

```
def list_notes(self) -> list[Note]:
    loaded = self._repository.list_all()
    visible: list[Note] = []
    for stored in loaded:
        if stored.is_private:
            if self._privacy is None:
                logger.error("Skipping private note %s: no privacy key", stored.id)
```

```
                continue
            try:
                plaintext = self._privacy.decrypt(stored.body)
            except PersistenceError as exc:
                logger.error("Skipping note %s: decrypt failed (%s)", stored.id, exc)
                continue
            visible.append(replace(stored, body=plaintext))
        else:
            visible.append(stored)
    visible.sort(key=lambda n: (n.modified_at, n.created_at), reverse=True)
    return visible
```

JsonFileRepository.list_all() was already doing the corrupt-file skip-and-log on disk-side errors (Sprint Zero). *NoteManager.list_notes()* adds the privacy-side skip-and-log: a note that cannot be decrypted does not crash the list, it is just hidden with an error logged. That is exactly the behavior the refined baseline asked for under FR-07.

Self-Review of Slice 2

I reviewed *NoteManager.list_notes()* in detail before declaring it done. Three things I noticed and fixed:

1. First draft caught a bare *Exception* around *decrypt*. I tightened it to *PersistenceError* so an unrelated bug (say, a *TypeError* from a corrupt payload) does not look like a decrypt failure. The narrower catch is consistent with *JsonFileRepository*, which only catches *OSError*, *JSONDecodeError*, *KeyError*, and *ValueError*.

2. First draft sorted by *modified_at* only. FR-08 specifies *created_at* as the tiebreaker, so I changed the key to *(modified_at, created_at)*. Two notes saved in the same millisecond would otherwise come back in undefined order on different file systems.

3. First draft mutated the loaded Note in place by reassigning *stored.body = plaintext*. That works but it edits the object the repository handed back, which would be confusing if anyone holds a reference to it. I switched to *dataclasses.replace*, which returns a fresh Note with the plaintext body and leaves the original alone. This matches the same pattern in *create_note* and keeps the function side-effect free.

Quality lesson. The pattern I keep falling into is 'make it work, then notice the failure mode'. For Slice 2 the issues I noticed before shipping (missing tiebreaker, broad except, in-place mutation) were all things a code reviewer would flag in five seconds. The lesson is to run each slice through the SPR-02 / NFR-02 lens explicitly before calling it done - 'what fails, what gets logged, can I still test this in isolation' - instead of waiting for the tests to surface it. Two of the three issues above had no failing test, just bad shape.

Traceability

Slice	Requirements realized	UML elements realized
Slice 1 *NoteManager.create_note*	FR-01 (create + validate + auto-id/timestamps), FR-04 (encrypt-on-save when *is_private*), FR-06 (UTC timestamps via Note factories), SPR-01 (Fernet before disk), SPR-02 (*ValidationError* / *PersistenceError* raised, never raw tracebacks)	*class-diagram.mmd*: *NoteManager*, *ValidationLayer*, *PrivacyService*, *JsonFileRepository*, *Note*, AstraNotesError family. *activity-diagram.mmd*: every step on the create-private path. *use-case-diagram.mmd*: UC1, UC4. *object-diagram.mmd*: note2 (the encrypted runtime instance).
Slice 2 *NoteManager.list_notes* + CLI option 2	FR-05 (load from disk, skip corrupt), FR-07 (decrypt-then-include, skip-decrypt-fail), FR-08 (sort by modified_at desc with created_at tiebreaker), NFR-02 (DI seam exercised), SPR-02 (errors logged, never surfaced as tracebacks)	*class-diagram.mmd*: *NoteManager.list_notes*, *JsonFileRepository.list_all*. New *activity-startup.mmd* and *activity-search.mmd* (added this week to close the Week 5.2 gaps). *use-case-diagram.mmd*: UC6, UC7.
CLI shell	NFR-02 (DI for *inp* / *out*), SPR-02 (AstraNotesError caught at the top of each menu action)	*deployment-diagram.mmd*: the *cli/* subgraph that was previously tagged 'Sprint 1' is now populated.

The deployment diagram's *cli/* node is no longer a placeholder - *astranotes/cli/app.py* exists and is the entry point the menu actually runs from.

In-Project Clean-up Shipped Alongside This Slice

These were the four 'Partially Traced' and 'Weakly Traced' items I flagged at the end of the Week 5.2 traceability matrix. They are now closed at the artifact level, not just on paper:

Artifact	Change
docs/uml/activity-search.mmd	New. FR-07 decrypt-then-match with skip-and-log and empty-collection short-circuit.
docs/uml/activity-startup.mmd	New. FR-05 first-launch directory creation plus the load loop's skip-and-log.
docs/uml/activity-toggle-privacy.mmd	New. FR-04 toggle-back-to-plaintext and decrypt-failure-keeps-encrypted branches.
docs/uml/deployment-diagram.mmd	Updated. SPR-04 license strings (Apache 2.0 / BSD for cryptography, MIT for pytest) now visible on the Deps nodes.

AI Reflection

I used GitHub Copilot in two modes this week.

Where Copilot helped. Inside the editor it suggested most of the boilerplate I expected - *from __future__ import annotations* at the top of each new module, the abstract-method docstrings I had already written once, the test fixtures (*tmp_path*, the scripted-input pattern for the CLI). For the new activity diagrams it gave me a usable Mermaid skeleton in a single completion; I kept the structure and only renamed nodes to match the method names already on the class diagram.

What I changed or rejected. Copilot's first pass at *NoteManager* had the orchestrator constructing its own *JsonFileRepository* and *PrivacyService* inside *__init__*. That breaks NFR-02 - there is no DI seam, the tests cannot substitute a fake, and *test_list_notes_skips_undecryptable_private_note* could not even be written. I rewrote the constructor to take all three collaborators as arguments, which is what the class diagram aggregation edges have shown since Week 4.1. Copilot also offered a 'delete note that can't be decrypted' path inside *list_notes*; I rejected it because FR-07 says 'exclude with an error logged', not 'delete from disk'. Silent data loss is the worst possible reading of SPR-02. For the CLI it suggested adding a 'Search' menu item at the same time as Slice 2 - I left that out because US-06 (search) is item 7 in the backlog and adding a UI affordance before the service-layer search method exists is exactly the kind of half-finished implementation the Working Agreement warns against.

The actual code that landed, the test list, and the traceability claims above are mine. Copilot accelerated typing, not design.